
CS 421 --- Combinator Parsing Activity

Name	Netid

Please write your name/netid legibly in dark ink. Hand in one copy per team. Do not staple or mangle the corners.

Purpose

Monadic combinator parsers work very similarly to the LL parsers we covered before, but the monadic interface manages the input stream for us. The resulting parsers are much easier to read and to write. Your goals are:

- Understand the types of the parser combinators.
- Explain the result of executing a parser.
- Explain the `<|>` combinator.
- Implement `many` and `many1`.

Part 1 --- The Types

```
0 newtype Parser t = Parser (String -> [(t,String)])
1 run (Parser p) = p
2
3 oneOf xx =
4   Parser (\inp -> case inp of
5     (s:ss) | s `elem` xx -> [(s,ss)]
6     otherwise             -> [])
7
8 sat pred =
9   Parser (\inp -> case inp of
10    (s:ss) | pred s    -> [(s,ss)]
11    otherwise          -> [])
12
13 p1 = run (oneOf "abc") "axy"
14 p2 = run (oneOf "abc") "xya"
```

The newtype is like data, but the resulting type has only one constructor, and it is optimized away by the compiler. We use it instead of simply saying `type Parser t = String -> [(t,String)]` because we can't declare a type as an instance, but we can declare a newtype as an instance.

Problem 1) Review the code with your team and come to a consensus on what each part is doing. What will be the values of `p1` and `p2`?

Problem 2) Can you write the function `digit` that parses a digit? Use `sat` to do this. For more of a challenge, have it return an actual integer.

Part 2 --- The Type Classes

```
0 instance Functor Parser where
1   fmap f (Parser p1) =
2       Parser (\inp -> [(f t, s) |
3                           (t,s) <- p1 inp])
4
5 instance Applicative Parser where
6   pure a = Parser (\inp -> [(a,inp)])
7   (Parser p1) <*> (Parser p2) =
8       Parser (\inp -> [(v1 v2, ss2) |
9                           (v1,ss1) <- p1 inp,
10                           (v2,ss2) <- p2 ss1])
11
12 instance Monad Parser where
13   (Parser p) >=> f =
14       Parser (\inp -> concat [run (f v) inp'
15                               | (v,inp') <- p inp])
16
17 data Exp = IntExp Integer
18         | PlusExp Exp Exp
19   deriving Show
20
21 p3 = run (IntExp <$> digit) "123"
22 p4 = run (PlusExp <$> getIntExp <*> getIntExp) "123"
23 p5 = do i1 <- getIntExp
24         i2 <- getIntExp
25         return (PlusExp i1 i2)
```

Problem 3) What is the value of `p3`? Trace through the evaluation and be sure everyone on your team understands how we got that result.

Problem 4) Write the function `getIntExp` that is like `digit` but encapsulates the digit in an `IntExp`.

Problem 5) What is the value of p4? Trace through the evaluation and be sure everyone on your team understands how we got that result.

Problem 6) What is the value of p5? Trace through the evaluation and be sure everyone on your team understands how we got that result.

Part 3 --- Choice, Many, Many1

```
0 (Parser p1) <|> (Parser p2) =  
1   Parser (\inp -> take 1 $ p1 inp ++ p2 inp)  
2  
3 string [] = Parser (\inp -> [([],inp)])  
4 string (s:ss) = do v <- char s  
5                   vv <- string ss  
6                   return $ v:vv  
7  
8 getPlusExp = do string "+"  
9               e1 <- getExp  
10              e2 <- getExp  
11              return (PlusExp e1 e2)  
12  
13 getExp = getIntExp  
14       <|> getPlusExp
```

Problem 7) Examine the code for <|>. How does it work? Hint: consider the cases that p1 succeeds, p1 fails but p2 succeeds, and both p1 and p2 fail.

Problem 8) Write the parsers `many p` and `many1 p` that take zero or more (for `many`) or one or more (for `many1`) repetitions of `p`.

Problem 9) The way we suggested writing `getIntExp` only works for a single digit. Can you make it work for multi-digit integers now?

Part 4 --- Precedence

Problem 10) Modify `PlusExp` to be infix, and add `TimesExp` as well. Stratify the grammar so that `TimesExp` has higher precedence than `PlusExp`.